FIRST MUTUAL LIFE ASSURANCE COMPANY

Requisition System

IT Guide

Version 1.0 | October 2026

Contents

1. Overview 3

2. Repository layout 4

3. Running locally 4

4. Configuration 4

5. Roles and permissions 5

6. Data model 5

Schema changes 6

7. Key behaviours 6

8. Deployment on Railway 7

Persistence: read this before going live 7

Release checklist 8

9. Operations 8

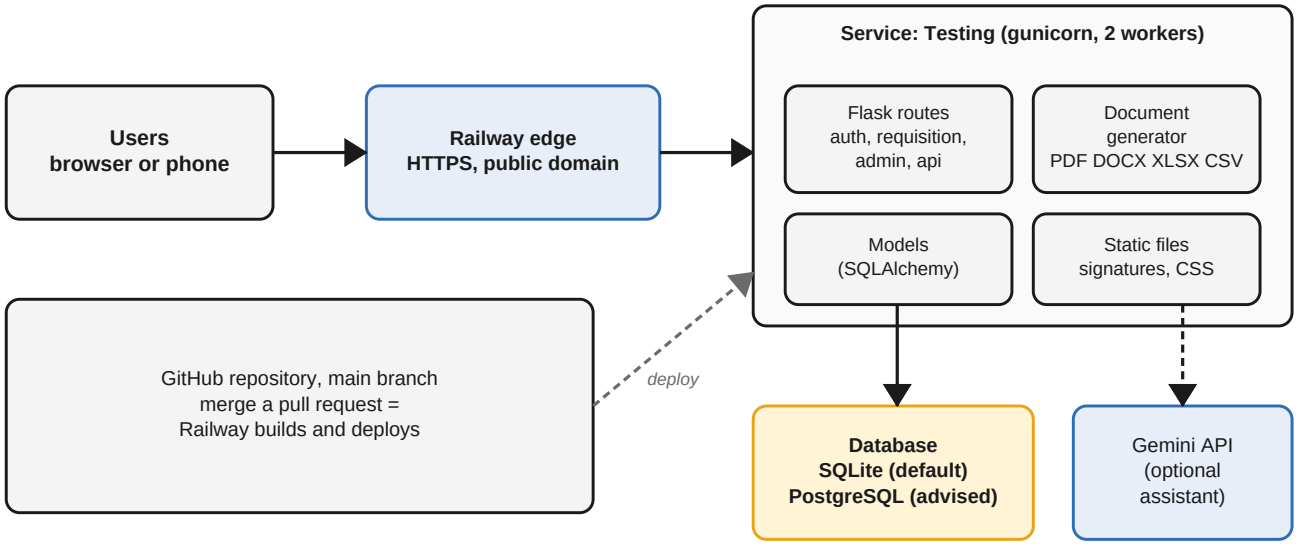
10. Security notes 8

11. Troubleshooting 9

For the people who deploy, configure, run and support the system. End users should read USER_MANUAL.md.

1. Overview

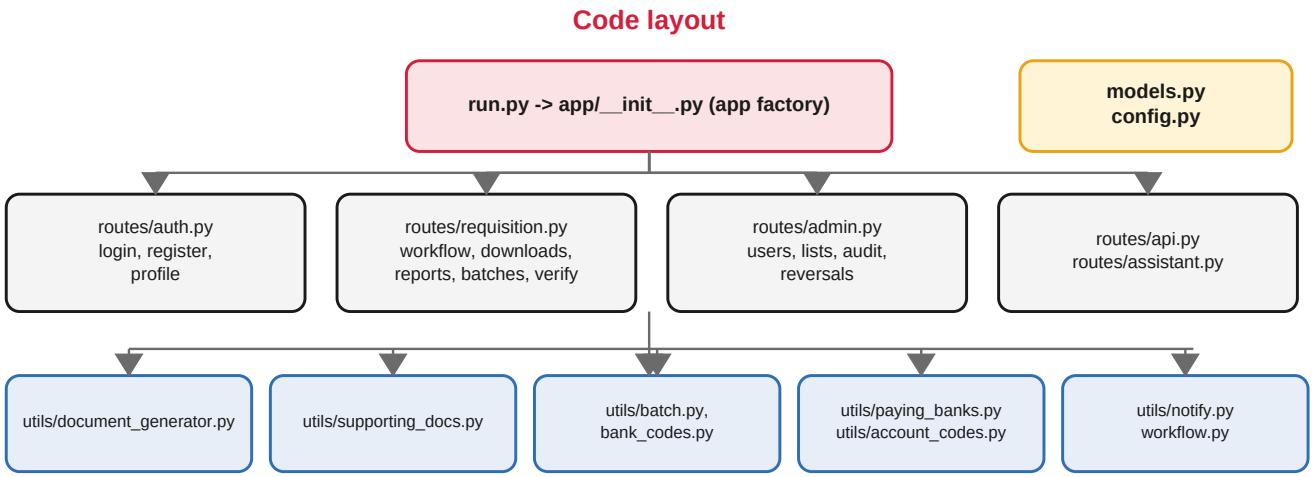
System architecture



A Flask web application for payment requisitions: submission, review, multi-level approval, signed and stamped documents, payment tracking, Stanbic batch files, reports and audit logs.

Part	Technology
Web framework	Flask 3.0, Flask-Login, Flask-WTF (CSRF)
Database	SQLAlchemy 2 (SQLite by default, PostgreSQL via `DATABASE_URL`)
Documents	reportlab (PDF), python-docx (Word), openpyxl (Excel/CSV templates), pypdf (merging), Pillow, qrcode
Server	gunicorn
Assistant	Google Gemini over HTTP (optional)
Hosting	Railway, deployed from the `main` branch

2. Repository layout




```
run.py                      entry point (exposes `app` for gunicorn)
create_superuser.py         interactive admin creation (alternative to env vars)
railway.toml                start command
requirements.txt
app/
  __init__.py               app factory, blueprints, startup migrations, superuser seeding
  config.py                 settings and approval thresholds
  models.py                 all database tables
  routes/
    auth.py                 login, registration, profile and signature
    requisition.py          requisition lifecycle, downloads, reports, batches, verify
    admin.py                users, audit logs, paying banks, account list, payment reversals
    api.py                  JSON for notifications and stats
    assistant.py            Gemini assistant
  utils/
    document_generator.py   PDF, DOCX, XLSX, CSV generation
    supporting_docs.py      validate, store and merge attachments
    batch.py, bank_codes.py Stanbic batch format and bank code lookup
    paying_banks.py, account_codes.py   Excel template and import parsers
    workflow.py, notify.py, decorators.py, reports.py, gemini.py, system_context.py
  templates/                Jinja templates (admin/, auth/, requisition/, components/)
  static/                   CSS, images, uploaded signatures (static/signatures)
```

3. Running locally

Python 3.12 is what is tested.

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export ADMIN_EMAIL=admin@example.com ADMIN_PASSWORD='choose-a-strong-one'
python run.py            # or: gunicorn run:app --bind 0.0.0.0:8080
```

On first start the tables are created and the admin user is seeded from the environment. Log in at `/auth/login`.

4. Configuration

All settings come from environment variables.

Variable	Purpose
`SECRET_KEY`	Signs sessions. **Set a long random value in production.** The code has a weak built-in default.

Variable	Purpose
`DATABASE_URL`	SQLAlchemy URL. Default `sqlite:///fml_requisition.db`. Use PostgreSQL in production.
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Superuser created or updated on startup.
`ADMIN_FIRST_NAME`, `ADMIN_LAST_NAME`, `ADMIN_EMPLOYEE_ID`, `ADMIN_DEPARTMENT`	Optional profile fields for that user.
`GEMINI_API_KEY`, `GEMINI_MODEL`	Enable the assistant. Without a key it logs a warning and the assistant is limited.
`GEMINI_KNOWLEDGE_MAX_AGE_HOURS`	How old the cached knowledge snapshot may be before refresh.
`FML_WEBSITE_URL`, `FML_WEBSITE_PATHS`, `FML_WEBSITE_CACHE_HOURS`	Company website pages the assistant reads.

Business rules live in `app/config.py` and `app/models.py`: approvals are 2, or 3 for USD 5,000 or more (`Requisition.calculate_required_approvals`). Upload limit is 16 MB.

5. Roles and permissions

Stored on the `users` table.

- `role`: `user`, `reviewer`, `approver` or `superuser` (admin).
- Flags: `can_initiate`, `can_review`, `can_approve`, `is_senior_approver`, and `is_active`.
- New self-registrations are inactive until an admin activates them and sets the flags.
- Admin pages require `role == superuser`. Batches are open to admins, reviewers and approvers. The third approval requires `is_senior_approver`.

6. Data model

Main tables and how they relate

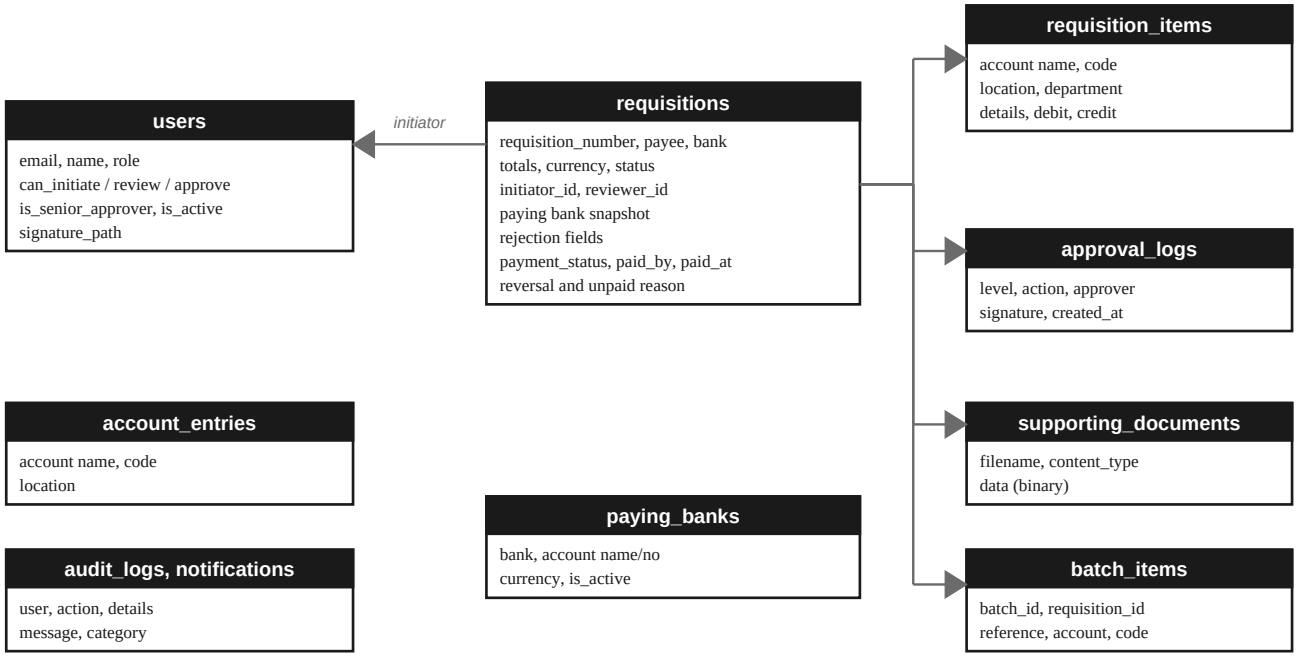


Table	Holds
`users`	Accounts, role flags, signature file name
`requisitions`	One per request: payee, bank, reason, totals, status, workflow actors, paying bank snapshot, rejection, payment status and reversal fields
`requisition_items`	Account lines: name, code, location, department, details, debit, credit
`approval_logs`	Each approve or reject action with level, signature and time
`supporting_documents`	Attachments stored as binary in the database
`paying_banks`	Admin-managed paying bank accounts
`account_entries`	Admin-managed account name, code, location list
`requisition_batches`, `batch_items`	Stanbic batch files and their lines
`notifications`, `audit_logs`	In-app notices and the action trail

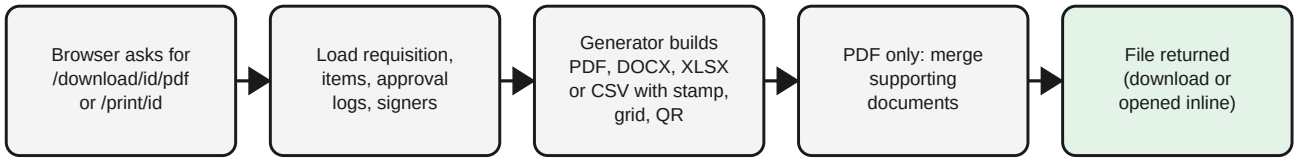
Status values: `draft`, `pending_review`, `pending_approval`, `partially_approved`, `approved`, `rejected`, `recalled`. Payment status is separate: `not_paid` (also when empty) or `paid`, meaningful once approved.

Schema changes

`db.create_all()` creates missing tables at startup. It does not alter existing ones, so new columns on `requisitions` and `batch_items` are added by hand-written `ALTER TABLE` lists in `app/__init__.py` (`_ensure_rejection_columns`, `_ensure_batch_columns`). When you add a column to those tables, add it to the matching list. Flask-Migrate is installed but not wired in.

7. Key behaviours

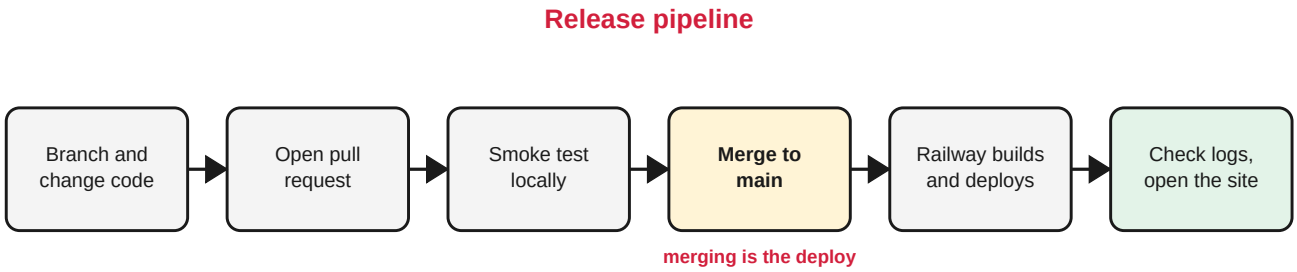
What happens on Download or Print



- **Workflow** (`routes/requisition.py`, `utils/workflow.py`): create, review, approve per level, reject returns to the initiator, recall is allowed until the first approval log exists. Rejected requisitions can be edited and resubmitted or deleted by the initiator.
- **Payment stamp**: shown when `status == approved`. Initiators mark Paid (`/mark-paid/<id>`). A reversal needs a reason plus an uploaded document (`/payment-reversal/<id>`), and only an admin clears it (`/admin/payment-reversals`).
- **Documents** (`utils/document_generator.py`): one generator drives PDF, DOCX, XLSX and CSV. The PDF download and `/print/<id>` are the same file, merged with supporting documents by `supporting_docs.merge_with_documents`.
- **Verification**: the QR encodes `/verify/<id>?code=<document code>`. The route is public. The document code is a hash of the number, totals, currency, status, approval count and approval log, so changing those invalidates old copies. Details are shown only when the code matches. The host in the QR comes from the request, so it is correct only if your proxy passes the real `Host` header.
- **Signatures**: image files under `app/static/signatures`, referenced by name in the database.

- **Dropdown lists**: paying banks and account entries are uploaded as `.xlsx` by an admin. Account list uploads ask for append or replace.
- **Stanbic batches**: `utils/batch.py` (format) and `utils/bank_codes.py` (bank to code and branch). To add a bank, append a row to `STANBIC_BANK_CODES` and any aliases to `_ALIASES`.

8. Deployment on Railway

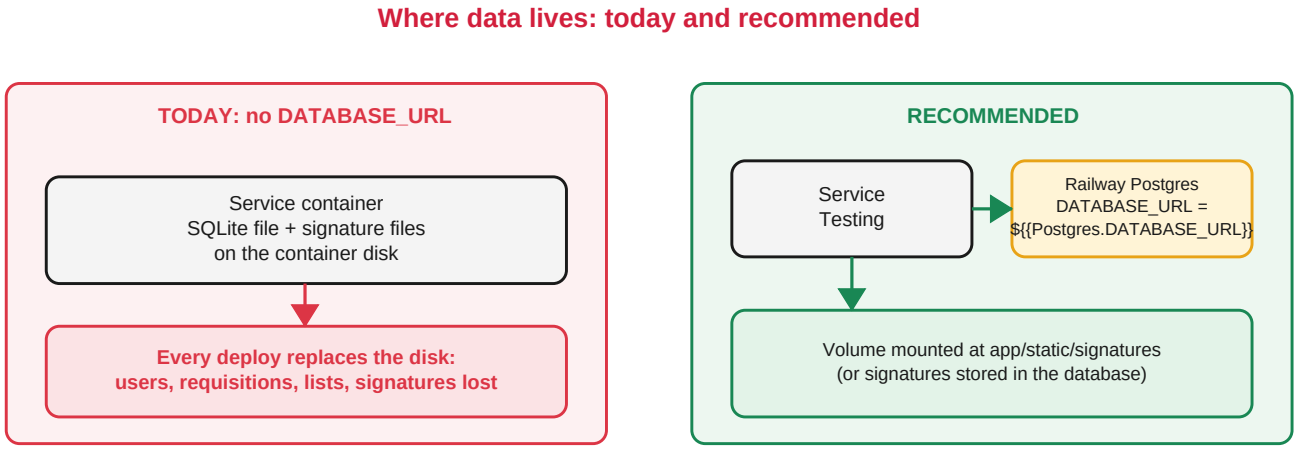


The service deploys from the `main` branch of `chiwonerwatylon-glitch/Testing`. Merging a pull request to `main` triggers a build and deploy. The start command is in `railway.toml`:

```
gunicorn run:app --bind 0.0.0.0:$PORT --workers 2 --timeout 120
```

Required variables: `SECRET_KEY`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`, and `DATABASE_URL` pointing at Postgres (see below). Optional: the Gemini and website variables.

Persistence: read this before going live



Without `DATABASE_URL` the app uses a SQLite file on the container disk. On Railway that disk is replaced on every deploy, so **all users, requisitions, uploaded lists and settings are lost at each deploy**. Uploaded signature files in `app/static/signatures` have the same problem.

Before real use:

1. Add a Railway Postgres database to the project and set `DATABASE_URL` on the service using the reference variable `${{Postgres.DATABASE_URL}}`. Tables are created on first start.
2. Give signatures a durable home. Attach a Railway volume mounted at `app/static/signatures`, or move signature storage into the database (as supporting documents already are).
3. Take regular database backups using Railway's backup tooling.

Release checklist

1. Branch from `main`, make the change, open a pull request.
2. Run a smoke test locally: log in, create and approve a requisition, download each format.
3. Merge. Watch the deployment logs until it shows success, then open the site.

9. Operations

Logs. Application logs go to stdout and appear in the Railway deployment logs. A line such as "Skipping knowledge refresh: GEMINI_API_KEY not configured" is informational.

Backups and restore. Back up the database and the signatures directory. Restore by pointing `DATABASE_URL` at the restored database and restoring the files.

Resetting the admin password. Change `ADMIN_PASSWORD` and redeploy. The startup seeding updates the superuser. `python create_superuser.py` creates one interactively.

Rotating `SECRET_KEY`. Changing it logs everyone out. Nothing else is affected.

Audit. The `audit_logs` table records logins, submissions, approvals, rejections, deletions, uploads, Paid and Not Paid changes. It is visible to admins under Audit logs. Keep it for your retention policy.

10. Security notes

- Set `SECRET_KEY` and a strong `ADMIN_PASSWORD`. Do not keep the defaults.
- Serve only over HTTPS. Railway provides this on its domains.
- CSRF protection is on for all forms.
- `/verify/<id>` is intentionally public but needs the matching document code. It never exposes bank account numbers. Requisition ids are sequential, so the code is what stops guessing.
- Attachments are limited to PDF, PNG and JPG, checked for validity, and capped at 16 MB per request.
- Self-registered users cannot act until an admin activates them.
- Treat `ADMIN_PASSWORD` and `GEMINI_API_KEY` as secrets. They live in Railway variables, not in the repository.

11. Troubleshooting

Symptom	Likely cause and fix
All data gone after a deploy	No `DATABASE_URL`. Configure Postgres (section 8).
Signature images missing after a deploy	Signature files were on the container disk. Use a volume or database storage.
New column error on an older database	Add the column to the `ALTER TABLE` list in `app/__init__.py` and redeploy.
Account list or paying banks upload refused	Wrong headings or missing required cells. The page names the rows. File must be `.xlsx`.
QR code points at the wrong host	The proxy is not forwarding the `Host` header. Fix the proxy or domain settings.
Assistant gives generic answers	`GEMINI_API_KEY` is missing or invalid.
Worker timeouts on big downloads	Large attachments make the merged PDF slow. The gunicorn timeout is 120 seconds. Raise it in `railway.toml` if needed.
Cannot log in as admin	Check `ADMIN_EMAIL` and `ADMIN_PASSWORD`, then redeploy to re-seed.